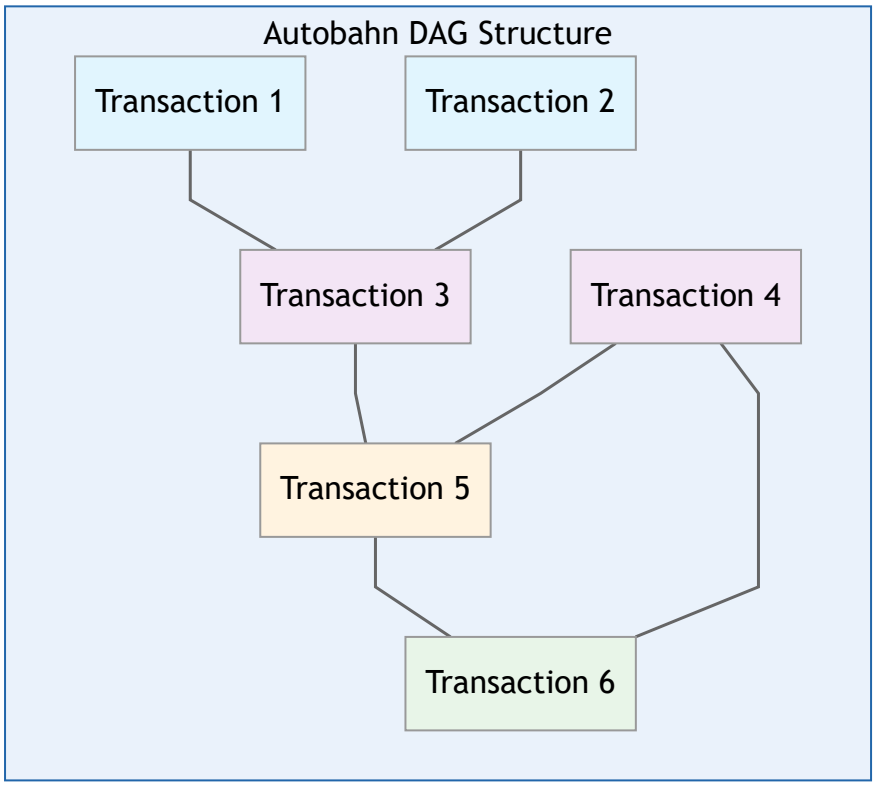
Autobahn DAG 관리자

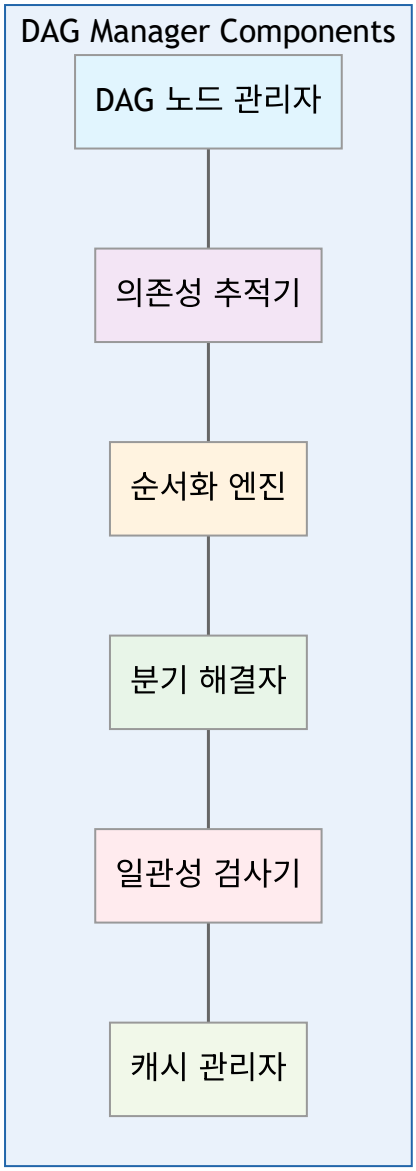
📊 DAG 기반 순서화 개념

Autobahn BFT의 DAG(Directed Acyclic Graph) 관리자는 **트랜잭션 간의 인과관계를 추적**하고 **결정론적 순서화**를 통해 모든 노드가 동일한 실행 순서를 보장합니다.

🔗 DAG 구조와 의존성



🏗️ DAG 관리자 아키텍처



🔧 핵심 컴포넌트 구현

1. DAG 노드 관리자

```
/// DAG 노드 관리자 - DAG 구조의 핵심 관리
pub struct DAGNodeManager {
/// 노드 저장소 (해시 -> 노드)
nodes: HashMap<Hash, DAGNode>,
/// 의존성 그래프 (자식 -> 부모들)
```

```
dependencies: HashMap<Hash, HashSet<Hash>>,

/// 역방향 의존성 (부모 -> 자식들)

reverse_dependencies: HashMap<Hash, HashSet<Hash>>,

/// 노드 메타데이터

node_metadata: HashMap<Hash, NodeMetadata>,

/// 가비지 컬렉션 관리

gc_manager: DAGGarbageCollector,

}



impl DAGNodeManager {

/// 새로운 DAG 노드 추가

pub  async  fn  add_node(&mut  self, node: DAGNode) -> Result<(), AutobahnError>

let  node_hash = node.hash();

// 1. 중복 노드 체크

if  self.nodes.contains_key(&node_hash) {

return Ok(()); // 이미 존재하는 노드

}

// 2. 부모 노드들 존재 확인

for  parent_hash  in &node.parents {

if !self.nodes.contains_key(parent_hash) {

return Err(AutobahnError::DAG(DAGError::MissingParent {

hash: parent_hash.as_bytes().try_into().unwrap()

}));
```

```
}
}
// 3. 순환 의존성 검사
if  self.would_create_cycle(&node_hash, &node.parents)? {
return Err(AutobahnError::DAG(DAGError::CircularDependency));
}
// 4. 노드 추가
self.nodes.insert(node_hash.clone(), node.clone());
// 5. 의존성 그래프 업데이트
self.dependencies.insert(node_hash.clone(), node.parents.clone());
for  parent_hash  in &node.parents {
self.reverse_dependencies
.entry(parent_hash.clone())
.or_insert_with(HashSet::new)
.insert(node_hash.clone());
}
// 6. 메타데이터 생성
let  metadata = NodeMetadata {
added_at: SystemTime::now(),
depth: self.calculate_node_depth(&node)?,
transaction_count: node.transactions.len(),
size_bytes: node.serialized_size(),
};
```

```
self.node_metadata.insert(node_hash.clone(), metadata);

// 7. 가비지 컬렉션 트리거 (필요시)

if  self.nodes.len() > self.gc_manager.gc_threshold() {

self.gc_manager.schedule_gc().await?;

}

Ok(())

}

/// 순환 의존성 검사

fn  would_create_cycle(&self, new_node: &Hash, parents: &HashSet<Hash>) -> Resul

// DFS를 사용한 순환 검사

let  mut  visited = HashSet::new();

let  mut  rec_stack = HashSet::new();

for  parent  in  parents {

if  self.has_cycle_dfs(parent, new_node, &mut  visited, &mut  rec_stack)? {

return Ok(true);

}

}

Ok(false)

}

/// DFS 기반 순환 검사

fn  has_cycle_dfs(

&self,

current: &Hash,
```

```
target: &Hash,
visited: &mut HashSet<Hash>,
rec_stack: &mut HashSet<Hash>,
) -> Result<bool, AutobahnError> {
if current == target {
return Ok(true);
}
if rec_stack.contains(current) {
return Ok(true);
}
if visited.contains(current) {
return Ok(false);
}
visited.insert(current.clone());
rec_stack.insert(current.clone());
if let Some(parents) = self.dependencies.get(current) {
for parent in parents {
if self.has_cycle_dfs(parent, target, visited, rec_stack)? {
return Ok(true);
}
}
}
rec_stack.remove(current);
```

```
Ok(false)
}
/// 노드 깊이 계산
fn calculate_node_depth(&self, node: &DAGNode) -> Result<u64, AutobahnError> {
if node.parents.is_empty() {
return Ok(0); // 루트 노드
}
let mut max_parent_depth = 0;
for parent_hash in &node.parents {
if let Some(parent_metadata) = self.node_metadata.get(parent_hash) {
max_parent_depth = max_parent_depth.max(parent_metadata.depth);
}
}
Ok(max_parent_depth + 1)
}
/// 노드 검색
pub fn get_node(&self, hash: &Hash) -> Option<&DAGNode> {
self.nodes.get(hash)
}
/// 자식 노드들 조회
pub fn get_children(&self, hash: &Hash) -> Vec<&DAGNode> {
if let Some(children_hashes) = self.reverse_dependencies.get(hash) {
children_hashes
```

```
.iter()
.filter_map(|child_hash| self.nodes.get(child_hash))
.collect()
} else {
Vec::new()
}
}
/// 부모 노드들 조회
pub fn get_parents(&self, hash: &Hash) -> Vec<&DAGNode> {
if let Some(node) = self.nodes.get(hash) {
node.parents
.iter()
.filter_map(|parent_hash| self.nodes.get(parent_hash))
.collect()
} else {
Vec::new()
}
}
}


/// DAG 노드 구조
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct DAGNode {
```

```rust
/// 노드 해시 (고유 식별자)
pub hash: Hash,
/// 부모 노드들의 해시
pub parents: HashSet<Hash>,
/// 포함된 트랜잭션들
pub transactions: Vec<Transaction>,
/// 노드 생성 시간
pub timestamp: SystemTime,
/// 노드 생성자
pub creator: NodeId,
/// 암호학적 서명
pub signature: Signature,
}

impl DAGNode {
/// 노드 해시 계산
pub fn hash(&self) -> Hash {
let mut hasher = Blake3Hasher::new();
hasher.update(&bincode::serialize(&self.parents).unwrap());
hasher.update(&bincode::serialize(&self.transactions).unwrap());
hasher.update(&bincode::serialize(&self.timestamp).unwrap());
hasher.update(self.creator.as_bytes());
Hash::new(hasher.finalize().to_vec())
```

```rust
}

/// 직렬화된 크기

pub fn serialized_size(&self) -> usize {

bincode::serialize(self).unwrap().len()

}

}



/// 노드 메타데이터

#[derive(Debug, Clone)]

pub struct NodeMetadata {

/// 추가된 시간

pub added_at: SystemTime,

/// DAG에서의 깊이

pub depth: u64,

/// 포함된 트랜잭션 수

pub transaction_count: usize,

/// 노드 크기 (바이트)

pub size_bytes: usize,

}
```

2. 의존성 추적기

```rust
/// 의존성 추적기 - 트랜잭션 간 인과관계 관리
```

```rust
pub struct DependencyTracker {
/// 트랜잭션 의존성 맵
transaction_dependencies: HashMap<TxHash, HashSet<TxHash>>,
/// 계정별 트랜잭션 순서
account_sequences: HashMap<AccountId, Vec<TxHash>>,
/// 리소스 접근 추적
resource_access_tracker: ResourceAccessTracker,
/// 의존성 해결 캐시
resolution_cache: LRUCache<TxHash, Vec<TxHash>>,
}

impl DependencyTracker {
/// 트랜잭션 의존성 분석
pub async fn analyze_dependencies(&mut self, transactions: &[Transaction]) -
let mut dependency_graph = DependencyGraph::new();
for tx in transactions {
// 1. 계정 기반 의존성 분석
let account_deps = self.analyze_account_dependencies(tx).await?;
// 2. 리소스 기반 의존성 분석
let resource_deps = self.resource_access_tracker.analyze_resource_dependencies(
// 3. 명시적 의존성 분석 (트랜잭션에 명시된 의존성)
let explicit_deps = self.analyze_explicit_dependencies(tx).await?;
// 4. 의존성 통합
```

```
let mut all_deps = HashSet::new();
all_deps.extend(account_deps);
all_deps.extend(resource_deps);
all_deps.extend(explicit_deps);
// 5. 의존성 그래프에 추가
dependency_graph.add_transaction(tx.hash(), all_deps);
// 6. 캐시 업데이트
self.update_dependency_cache(tx.hash(), &all_deps).await?;
}
Ok(dependency_graph)
}
/// 계정 기반 의존성 분석
async fn analyze_account_dependencies(&mut self, tx: &Transaction) -> Result<
let mut dependencies = HashSet::new();
// 트랜잭션이 접근하는 계정들
let accessed_accounts = tx.get_accessed_accounts();
for account in accessed_accounts {
// 해당 계정의 이전 트랜잭션들과 의존성 생성
if let Some(account_txs) = self.account_sequences.get(&account) {
// 읽기-쓰기, 쓰기-읽기, 쓰기-쓰기 의존성
for prev_tx_hash in account_txs {
if self.has_dependency(prev_tx_hash, &tx.hash(), &account).await? {
dependencies.insert(prev_tx_hash.clone());
```

```
}
}
}
// 계정 시퀀스 업데이트
self.account_sequences
.entry(account)
.or_insert_with(Vec::new)
.push(tx.hash());
}
Ok(dependencies)
}
/// 의존성 존재 여부 확인
async fn has_dependency(&self, prev_tx: &TxHash, current_tx: &TxHash, account:
// 캐시에서 먼저 확인
if let Some(cached_deps) = self.resolution_cache.get(current_tx) {
return Ok(cached_deps.contains(prev_tx));
}
// 실제 의존성 분석
let prev_transaction = self.get_transaction(prev_tx).await?;
let current_transaction = self.get_transaction(current_tx).await?;
// 읽기-쓰기 의존성 (RAW)
if prev_transaction.writes_to_account(account) && current_transaction.reads_fro
return Ok(true);
```

```
}
// 쓰기-읽기 의존성 (WAR)
if  prev_transaction.reads_from_account(account) && current_transaction.writes_t
return Ok(true);
}
// 쓰기-쓰기 의존성 (WAW)
if  prev_transaction.writes_to_account(account) && current_transaction.writes_to
return Ok(true);
}
Ok(false)
}
}


/// 리소스 접근 추적기
pub  struct ResourceAccessTracker {
/// 리소스별 접근 히스토리
resource_access_history: HashMap<ResourceId, Vec<ResourceAccess>>,
/// 락 관리자
lock_manager: ResourceLockManager,
}


impl ResourceAccessTracker {
/// 리소스 기반 의존성 분석
```

```
pub  async  fn  analyze_resource_dependencies(&mut  self, tx: &Transaction) -> R
let  mut  dependencies = HashSet::new();
let  resources = tx.get_accessed_resources();
for  resource  in  resources {
// 리소스에 대한 이전 접근들과 의존성 확인
if  let Some(access_history) = self.resource_access_history.get(&resource.id) {
for  prev_access  in  access_history {
if  self.conflicts_with_access(&resource, prev_access) {
dependencies.insert(prev_access.transaction_hash.clone());
}
}
}
// 새로운 접근 기록
let  new_access = ResourceAccess {
resource_id: resource.id.clone(),
access_type: resource.access_type,
transaction_hash: tx.hash(),
timestamp: SystemTime::now(),
};
self.resource_access_history
.entry(resource.id.clone())
.or_insert_with(Vec::new)
.push(new_access);
```

```
}
Ok(dependencies)
}
/// 리소스 접근 충돌 확인
fn  conflicts_with_access(&self, current: &ResourceAccess, previous: &ResourceAc
match (previous.access_type, current.access_type) {
(AccessType::Read, AccessType::Read) => false, // 읽기-읽기는 충돌 없음
(AccessType::Read, AccessType::Write) => true, // 읽기-쓰기 충돌
(AccessType::Write, AccessType::Read) => true, // 쓰기-읽기 충돌
(AccessType::Write, AccessType::Write) => true, // 쓰기-쓰기 충돌
}
}
}


/// 의존성 그래프
#[derive(Debug, Clone)]
pub  struct DependencyGraph {
/// 노드들 (트랜잭션 해시)
nodes: HashSet<TxHash>,
/// 간선들 (의존성)
edges: HashMap<TxHash, HashSet<TxHash>>,
/// 역방향 간선들
reverse_edges: HashMap<TxHash, HashSet<TxHash>>,
```

```
}

impl DependencyGraph {
pub fn new() -> Self {
Self {
nodes: HashSet::new(),
edges: HashMap::new(),
reverse_edges: HashMap::new(),
}
}
/// 트랜잭션과 의존성 추가
pub fn add_transaction(&mut self, tx_hash: TxHash, dependencies: HashSet<TxHa
self.nodes.insert(tx_hash.clone());
for dep in &dependencies {
self.nodes.insert(dep.clone());
// 의존성 간선 추가 (dep -> tx_hash)
self.edges
.entry(dep.clone())
.or_insert_with(HashSet::new)
.insert(tx_hash.clone());
// 역방향 간선 추가 (tx_hash <- dep)
self.reverse_edges
.entry(tx_hash.clone())
```

```
.or_insert_with(HashSet::new)
.insert(dep.clone());
}
}
/// 토폴로지 정렬
pub fn topological_sort(&self) -> Result<Vec<TxHash>, AutobahnError> {
let mut in_degree: HashMap<TxHash, usize> = HashMap::new();
let mut queue = VecDeque::new();
let mut result = Vec::new();
// 1. 진입 차수 계산
for node in &self.nodes {
let degree = self.reverse_edges.get(node).map_or(0, |deps| deps.len());
in_degree.insert(node.clone(), degree);
if degree == 0 {
queue.push_back(node.clone());
}
}
// 2. 토폴로지 정렬 수행
while let Some(current) = queue.pop_front() {
result.push(current.clone());
// 현재 노드의 후속 노드들 처리
if let Some(successors) = self.edges.get(&current) {
for successor in successors {
```

```
if  let Some(degree) = in_degree.get_mut(successor) {
*degree -= 1;
if *degree == 0 {
queue.push_back(successor.clone());
}
}
}
}
}
// 3. 순환 검사
if  result.len() != self.nodes.len() {
return Err(AutobahnError::DAG(DAGError::CircularDependency));
}
Ok(result)
}
}
```

3. 순서화 엔진

```
/// 순서화 엔진 - 결정론적 트랜잭션 순서 생성
pub  struct OrderingEngine {
/// 의존성 추적기
```

```
dependency_tracker: DependencyTracker,

/// 분기 해결자

fork_resolver: ForkResolver,

/// 순서화 알고리즘

ordering_algorithm: OrderingAlgorithm,

/// 순서화 캐시

ordering_cache: OrderingCache,

}



impl OrderingEngine {

/// 트랜잭션 순서화

pub  async  fn  order_transactions(&mut  self, dag_nodes: &[DAGNode]) -> Result<

// 1. 모든 트랜잭션 수집

let  mut  all_transactions = Vec::new();

for  node  in  dag_nodes {

all_transactions.extend(node.transactions.clone());

}

// 2. 의존성 분석

let  dependency_graph = self.dependency_tracker.analyze_dependencies(&all_transa

// 3. 기본 토폴로지 정렬

let  mut  ordered_hashes = dependency_graph.topological_sort()?;

// 4. 분기 해결 (동일한 우선순위의 트랜잭션들)

ordered_hashes = self.fork_resolver.resolve_ordering_conflicts(ordered_hashes, &
```

```
// 5. 최종 순서화 적용
let ordered_transactions = self.apply_final_ordering(ordered_hashes, &all_trans
// 6. 순서화 결과 캐시
self.cache_ordering_result(&ordered_transactions).await?;
Ok(ordered_transactions)
}
/// 최종 순서화 적용
async fn apply_final_ordering(&self, ordered_hashes: Vec<TxHash>, transactions
let tx_map: HashMap<TxHash, Transaction> = transactions
.iter()
.map(|tx| (tx.hash(), tx.clone()))
.collect();
let mut ordered_transactions = Vec::new();
for tx_hash in ordered_hashes {
if let Some(tx) = tx_map.get(&tx_hash) {
ordered_transactions.push(tx.clone());
}
}
Ok(ordered_transactions)
}
/// 병렬 순서화 (성능 최적화)
pub async fn order_transactions_parallel(&mut self, dag_nodes: &[DAGNode]) -
// 1. 노드들을 독립적인 그룹으로 분할
```

```
let independent_groups = self.partition_independent_groups(dag_nodes).await?;
// 2. 각 그룹을 병렬로 순서화
let mut group_futures = Vec::new();
for group in independent_groups {
let mut engine_clone = self.clone();
let future = tokio::spawn(async move {
engine_clone.order_transactions(&group).await
});
group_futures.push(future);
}
// 3. 결과 수집 및 병합
let mut all_ordered_transactions = Vec::new();
for future in group_futures {
let group_result = future.await.map_err(|e| AutobahnError::Performance(e.to_str
all_ordered_transactions.extend(group_result);
}
// 4. 그룹 간 순서화
let final_ordered = self.merge_group_orderings(all_ordered_transactions).await?
Ok(final_ordered)
}
/// 독립적인 그룹 분할
async fn partition_independent_groups(&self, dag_nodes: &[DAGNode]) -> Result<
let mut groups = Vec::new();
```

```
let mut visited = HashSet::new();
for node in dag_nodes {
if !visited.contains(&node.hash) {
let group = self.find_connected_component(node, dag_nodes, &mut visited).await
groups.push(group);
}
}
Ok(groups)
}
/// 연결된 컴포넌트 찾기
async fn find_connected_component(&self, start_node: &DAGNode, all_nodes: &[DA
let mut component = Vec::new();
let mut stack = vec![start_node.clone()];
while let Some(current) = stack.pop() {
if visited.insert(current.hash.clone()) {
component.push(current.clone());
// 연결된 노드들 찾기
for node in all_nodes {
if !visited.contains(&node.hash) {
if self.are_nodes_connected(&current, node).await? {
stack.push(node.clone());
}
}
```

```
}
}
}
Ok(component)
}
/// 노드 간 연결성 확인
async fn are_nodes_connected(&self, node1: &DAGNode, node2: &DAGNode) -> Resul
// 직접적인 부모-자식 관계
if node1.parents.contains(&node2.hash) || node2.parents.contains(&node1.hash) {
return Ok(true);
}
// 트랜잭션 간 의존성
for tx1 in &node1.transactions {
for tx2 in &node2.transactions {
if self.dependency_tracker.has_dependency(&tx1.hash(), &tx2.hash()).await? {
return Ok(true);
}
}
}
Ok(false)
}
}
```

```rust
/// 순서화 알고리즘 종류
#[derive(Debug, Clone)]
pub enum OrderingAlgorithm {
/// 기본 토폴로지 정렬
TopologicalSort,
/// 우선순위 기반 정렬
PriorityBased,
/// 타임스탬프 기반 정렬
TimestampBased,
/// 하이브리드 정렬
Hybrid,
}

/// 순서화 캐시
pub struct OrderingCache {
/// 순서화 결과 캐시
result_cache: LRUCache<Vec<Hash>, Vec<TxHash>>,
/// 의존성 캐시
dependency_cache: LRUCache<TxHash, HashSet<TxHash>>,
/// 캐시 통계
cache_stats: CacheStatistics,
}
```

```
impl OrderingCache {
/// 순서화 결과 캐시 조회
pub fn get_cached_ordering(&mut self, dag_hashes: &[Hash]) -> Option<Vec<TxHa
self.cache_stats.record_access();
if let Some(result) = self.result_cache.get(&dag_hashes.to_vec()) {
self.cache_stats.record_hit();
Some(result.clone())
} else {
self.cache_stats.record_miss();
None
}
}
/// 순서화 결과 캐시 저장
pub fn cache_ordering_result(&mut self, dag_hashes: Vec<Hash>, ordered_txs: V
self.result_cache.put(dag_hashes, ordered_txs);
}
}
```

4. 분기 해결자

```
/// 분기 해결자 - 동일한 우선순위 트랜잭션들의 결정론적 순서화
pub struct ForkResolver {
/// Tie-breaking 규칙들
```

```
tie_breaking_rules: Vec<TieBreakingRule>,
/// 해시 비교기
hash_comparator: HashComparator,
/// 타임스탬프 비교기
timestamp_comparator: TimestampComparator,
/// 우선순위 계산기
priority_calculator: PriorityCalculator,
}

impl ForkResolver {
/// 순서화 충돌 해결
pub  async  fn  resolve_ordering_conflicts(&mut  self, ordered_hashes: Vec<TxHas
let  mut  resolved_order = Vec::new();
let  mut  remaining_hashes = ordered_hashes;
while !remaining_hashes.is_empty() {
// 1. 현재 처리 가능한 트랜잭션들 찾기
let  ready_transactions = self.find_ready_transactions(&remaining_hashes, depend
if  ready_transactions.is_empty() {
return Err(AutobahnError::DAG(DAGError::OrderingConflict));
}
// 2. 단일 트랜잭션인 경우 바로 추가
if  ready_transactions.len() == 1 {
let  tx_hash = ready_transactions[0].clone();
```

```
resolved_order.push(tx_hash.clone());
remaining_hashes.retain(|h| h != &tx_hash);
continue;
}
// 3. 여러 트랜잭션이 동시에 준비된 경우 tie-breaking 적용
let ordered_ready = self.apply_tie_breaking_rules(ready_transactions).await?;
// 4. 순서대로 추가
for tx_hash in ordered_ready {
resolved_order.push(tx_hash.clone());
remaining_hashes.retain(|h| h != &tx_hash);
}
}
Ok(resolved_order)
}
/// 처리 준비된 트랜잭션들 찾기
fn find_ready_transactions(&self, remaining: &[TxHash], dependency_graph: &Depe
let mut ready = Vec::new();
let completed_set: HashSet<_> = completed.iter().collect();
for tx_hash in remaining {
// 모든 의존성이 완료되었는지 확인
if let Some(dependencies) = dependency_graph.reverse_edges.get(tx_hash) {
let all_deps_completed = dependencies.iter().all(|dep| completed_set.contains(d
if all_deps_completed {
```

```
ready.push(tx_hash.clone());
}
} else {
// 의존성이 없는 트랜잭션
ready.push(tx_hash.clone());
}
}
Ok(ready)
}
/// Tie-breaking 규칙 적용
async fn apply_tie_breaking_rules(&mut self, transactions: Vec<TxHash>) -> Re
let mut sorted_transactions = transactions;
// 각 tie-breaking 규칙을 순서대로 적용
for rule in &self.tie_breaking_rules {
sorted_transactions = self.apply_single_rule(sorted_transactions, rule).await?;
}
Ok(sorted_transactions)
}
/// 단일 tie-breaking 규칙 적용
async fn apply_single_rule(&mut self, mut transactions: Vec<TxHash>, rule: &
match rule {
TieBreakingRule::HashBased => {
// 해시 기준 정렬 (결정론적)
```

```
transactions.sort_by(|a, b| self.hash_comparator.compare(a, b));
}
TieBreakingRule::TimestampBased => {
// 타임스탬프 기준 정렬
transactions.sort_by(|a, b| self.timestamp_comparator.compare(a, b));
}
TieBreakingRule::PriorityBased => {
// 우선순위 기준 정렬
transactions.sort_by(|a, b| {
let  priority_a = self.priority_calculator.calculate_priority(a);
let  priority_b = self.priority_calculator.calculate_priority(b);
priority_b.cmp(&priority_a) // 높은 우선순위가 먼저
});
}
TieBreakingRule::GasFeeBased => {
// 가스 수수료 기준 정렬
transactions.sort_by(|a, b| {
let  fee_a = self.get_transaction_gas_fee(a);
let  fee_b = self.get_transaction_gas_fee(b);
fee_b.cmp(&fee_a) // 높은 수수료가 먼저
});
}
TieBreakingRule::AccountNonceBased => {
```

```
// 계정 nonce 기준 정렬
transactions.sort_by(|a, b| {
let  nonce_a = self.get_transaction_nonce(a);
let  nonce_b = self.get_transaction_nonce(b);
nonce_a.cmp(&nonce_b) // 낮은 nonce가 먼저
});
}
}
Ok(transactions)
}
}

/// Tie-breaking 규칙 종류
#[derive(Debug, Clone)]
pub  enum TieBreakingRule {
/// 해시 기반 (결정론적)
HashBased,
/// 타임스탬프 기반
TimestampBased,
/// 우선순위 기반
PriorityBased,
/// 가스 수수료 기반
GasFeeBased,
```

```rust
/// 계정 nonce 기반
AccountNonceBased,
}

/// 해시 비교기
pub struct HashComparator;

impl HashComparator {
/// 해시 기반 비교 (결정론적)
pub fn compare(&self, hash_a: &TxHash, hash_b: &TxHash) -> std::cmp::Ordering
hash_a.as_bytes().cmp(hash_b.as_bytes())
}
}

/// 우선순위 계산기
pub struct PriorityCalculator {
/// 우선순위 가중치
priority_weights: PriorityWeights,
}

impl PriorityCalculator {
/// 트랜잭션 우선순위 계산
pub fn calculate_priority(&self, tx_hash: &TxHash) -> u64 {
```

```
let tx = self.get_transaction(tx_hash);
let mut priority = 0u64;
// 가스 수수료 기여도
priority += (tx.gas_fee() as f64 * self.priority_weights.gas_fee_weight) as u64;
// 트랜잭션 크기 기여도 (작을수록 높은 우선순위)
let size_penalty = (tx.size() as f64 * self.priority_weights.size_penalty_weigh
priority = priority.saturating_sub(size_penalty);
// 계정 nonce 기여도 (낮을수록 높은 우선순위)
let nonce_bonus = (1000u64).saturating_sub(tx.nonce());
priority += (nonce_bonus as f64 * self.priority_weights.nonce_weight) as u64;
// 트랜잭션 타입 기여도
priority += match tx.transaction_type() {
TransactionType::Transfer => 100,
TransactionType::ContractCall => 200,
TransactionType::ContractDeploy => 300,
TransactionType::System => 1000,
};
priority
}
}

/// 우선순위 가중치
#[derive(Debug, Clone)]
```

```rust
pub struct PriorityWeights {
pub gas_fee_weight: f64,
pub size_penalty_weight: f64,
pub nonce_weight: f64,
}

impl Default for PriorityWeights {
fn default() -> Self {
Self {
gas_fee_weight: 1.0,
size_penalty_weight: 0.1,
nonce_weight: 0.5,
}
}
}
```

5. 일관성 검사기

```rust
/// 일관성 검사기 - DAG 구조와 순서화 결과의 일관성 보장
pub struct ConsistencyChecker {
/// 구조적 일관성 검사기
structural_checker: StructuralConsistencyChecker,
```

```
/// 의미적 일관성 검사기
semantic_checker: SemanticConsistencyChecker,
/// 암호학적 검증기
cryptographic_verifier: CryptographicVerifier,
}


impl ConsistencyChecker {
/// 전체 일관성 검사
pub  async  fn  check_consistency(&mut  self, dag: &DAGNodeManager, ordering: &[
let  mut  report = ConsistencyReport::new();
// 1. 구조적 일관성 검사
let  structural_result = self.structural_checker.check_dag_structure(dag).await?
report.add_structural_result(structural_result);
// 2. 의미적 일관성 검사
let  semantic_result = self.semantic_checker.check_transaction_semantics(orderin
report.add_semantic_result(semantic_result);
// 3. 암호학적 검증
let  crypto_result = self.cryptographic_verifier.verify_signatures_and_hashes(da
report.add_cryptographic_result(crypto_result);
// 4. 순서화 일관성 검사
let  ordering_result = self.check_ordering_consistency(dag, ordering).await?;
report.add_ordering_result(ordering_result);
Ok(report)
```

```
}

/// 순서화 일관성 검사

async fn check_ordering_consistency(&self, dag: &DAGNodeManager, ordering: &[T

let mut violations = Vec::new();

// 1. 의존성 순서 검사

for (i, tx) in ordering.iter().enumerate() {

let dependencies = self.get_transaction_dependencies(tx).await?;

for dep_hash in dependencies {

// 의존성이 현재 트랜잭션보다 뒤에 나오는지 확인

if let Some(dep_position) = ordering.iter().position(|t| t.hash() == dep_hash)

if dep_position > i {

violations.push(OrderingViolation::DependencyOrderViolation {

transaction: tx.hash(),

dependency: dep_hash,

tx_position: i,

dep_position,

});

}

}

}

}

// 2. DAG 구조와 순서 일치성 검사

for node in dag.get_all_nodes() {
```

```
for tx in &node.transactions {
if let Some(tx_position) = ordering.iter().position(|t| t.hash() == tx.hash())
// 부모 노드의 모든 트랜잭션이 현재 트랜잭션보다 앞에 있는지 확인
for parent_hash in &node.parents {
if let Some(parent_node) = dag.get_node(parent_hash) {
for parent_tx in &parent_node.transactions {
if let Some(parent_tx_position) = ordering.iter().position(|t| t.hash() == pare
if parent_tx_position >= tx_position {
violations.push(OrderingViolation::DAGOrderViolation {
child_tx: tx.hash(),
parent_tx: parent_tx.hash(),
child_position: tx_position,
parent_position: parent_tx_position,
});
}
}
}
}
}
}
}
Ok(OrderingConsistencyResult {
```

```
is_consistent: violations.is_empty(),
violations,
})
}
}

/// 구조적 일관성 검사기
pub struct StructuralConsistencyChecker;

impl StructuralConsistencyChecker {
/// DAG 구조 검사
pub async fn check_dag_structure(&mut self, dag: &DAGNodeManager) -> Result<
let mut issues = Vec::new();
// 1. 순환 참조 검사
if let Err(_) = self.check_for_cycles(dag) {
issues.push(StructuralIssue::CyclicDependency);
}
// 2. 고아 노드 검사
let orphan_nodes = self.find_orphan_nodes(dag);
if !orphan_nodes.is_empty() {
issues.push(StructuralIssue::OrphanNodes(orphan_nodes));
}
// 3. 무효한 참조 검사
```

```
let invalid_refs = self.find_invalid_references(dag);
if !invalid_refs.is_empty() {
issues.push(StructuralIssue::InvalidReferences(invalid_refs));
}
Ok(StructuralConsistencyResult {
is_valid: issues.is_empty(),
issues,
})
}
/// 순환 참조 검사
fn check_for_cycles(&self, dag: &DAGNodeManager) -> Result<(), AutobahnError> {
let mut visited = HashSet::new();
let mut rec_stack = HashSet::new();
for node_hash in dag.get_all_node_hashes() {
if !visited.contains(&node_hash) {
self.dfs_cycle_check(&node_hash, dag, &mut visited, &mut rec_stack)?;
}
}
Ok(())
}
/// DFS 기반 순환 검사
fn dfs_cycle_check(
&self,
```

```
node_hash: &Hash,
dag: &DAGNodeManager,
visited: &mut HashSet<Hash>,
rec_stack: &mut HashSet<Hash>,
) -> Result<(), AutobahnError> {
visited.insert(node_hash.clone());
rec_stack.insert(node_hash.clone());
if let Some(node) = dag.get_node(node_hash) {
for parent_hash in &node.parents {
if !visited.contains(parent_hash) {
self.dfs_cycle_check(parent_hash, dag, visited, rec_stack)?;
} else if rec_stack.contains(parent_hash) {
return Err(AutobahnError::DAG(DAGError::CircularDependency));
}
}
}
rec_stack.remove(node_hash);
Ok(())
}
}

/// 일관성 검사 보고서
#[derive(Debug)]
```

```rust
pub struct ConsistencyReport {
pub structural_result: Option<StructuralConsistencyResult>,
pub semantic_result: Option<SemanticConsistencyResult>,
pub cryptographic_result: Option<CryptographicConsistencyResult>,
pub ordering_result: Option<OrderingConsistencyResult>,
pub overall_consistent: bool,
}


impl ConsistencyReport {
pub fn new() -> Self {
Self {
structural_result: None,
semantic_result: None,
cryptographic_result: None,
ordering_result: None,
overall_consistent: false,
}
}
pub fn is_consistent(&self) -> bool {
self.structural_result.as_ref().map_or(false, |r| r.is_valid) &&
self.semantic_result.as_ref().map_or(false, |r| r.is_valid) &&
self.cryptographic_result.as_ref().map_or(false, |r| r.is_valid) &&
self.ordering_result.as_ref().map_or(false, |r| r.is_consistent)
```

```
}

}
```

📊 성능 최적화

DAG 캐싱 전략

```
/// DAG 캐시 관리자
pub struct DAGCacheManager {
/// 노드 캐시
node_cache: LRUCache<Hash, DAGNode>,
/// 의존성 캐시
dependency_cache: LRUCache<Hash, HashSet<Hash>>,
/// 순서화 캐시
ordering_cache: LRUCache<Vec<Hash>, Vec<TxHash>>,
/// 캐시 통계
cache_stats: CacheStatistics,
}

impl DAGCacheManager {
/// 캐시 최적화
pub async fn optimize_cache(&mut self) -> Result<(), AutobahnError> {
// 1. 사용 빈도 기반 캐시 조정
```

```
self.adjust_cache_sizes_by_usage().await?;

// 2. 만료된 항목 정리

self.cleanup_expired_entries().await?;

// 3. 캐시 압축

self.compress_cache_entries().await?;

Ok(())

}

}
```

🎯 결론

Autobahn DAG 관리자는:

- ✅ **결정론적 순서화**: 모든 노드가 동일한 실행 순서 보장
- ✅ **효율적인 의존성 추적**: 트랜잭션 간 인과관계 정확한 관리
- ✅ **분기 해결**: 충돌 시 일관된 해결 방법 제공
- ✅ **성능 최적화**: 캐싱과 병렬 처리를 통한 고성능 달성

다음 문서에서는 순수 시스템 아키텍처에 대해 알아보겠습니다.